<u>תמר בוזנח – הסבר על הfs</u>

fs שלי מורכב מ4,096 בלוקים כאשר כל בלוק בגודל של 256 bytes, משמע כל הfs בגודל של 1mb (4096*256).

<u>חלוקת הבלוקים שלי</u>

Block 0: superblock
Blocks 1-4: block table
Blocks 5-2053: inode table
Blocks 2054-4096: our data

<u>Superblock</u>

superblockהוא ההתחלה של המקום שאני שומרת בו את כל המערכת קבצים שלי (אינדקס 0), ההתחלה תהיה רצף ידוע מראש שמסמן את ההתחלה של המערכת קבצים (אצלי זה TBFS), הגודל של כל בלוק, וכמות הבלוקים במערכת קבצים. הגודל של כל המערכת קבצים זה כמות הבלוקים כפול הגודל של כל בלוק.

<u>Blocktable</u>

למרות ההמלצה בקובץ של לציין בכל תחילת בלוק האם הבלוק תפוס או לא, החלטתי להשתמש במקום בblocktable שהחלטתי לממש אותו במערך של intים בגודל 2046 שבו כל אנטרי במערך מייצג את הסטטוס של הבלוק (0 – תפוס, 1- פנוי).

<u>איך יצרתי את טבלת הinodes?</u>

```
struct NodeStruct {
    std::string filename;   // max filename can be 10, each char is 1 byte so all filename is 10 bytes
    int typeFile;  // 4 bytes
    int sizeFile;  // 4 bytes
    int ptrArray[6]; // 4 * 6 = 24 bytes
};
NodeStruct myInode[INT_BLOCK_SIZE] = {};   // init all to 0
```

בקוד שלי יצרתי מערך של structים בגודל של 256 (גודל של בלוק בודד) שהוא מייצג את הinode

Filename: שם הקובץ
typeFile: מזהה סוג: יקבל את הערך 1 אם מדובר בקובץ, 2 – אם מדובר בתיקייה (אם אממש את זה), 0- הקובץ\התיקייה מחוקים.
sizeFile: הגודל של הקובץ – יסייע כשנחפש את הקובץ לאחר פרגמנטציה.
ptrArray: סט של שישה pointerים על בלוקים – כל "pointer" הוא מספר block שמשמש לשמירת המידע שנמצא בקובץ\בתיקייה.

הוא נכתב בבלוק החמישי עד בלוק 2053 (לכל בלוק שכותבים אליו צריך להיות בלוק שמשמש לשמירת המידע עליו בinode).

הגודל של הstruct הוא 42 bytes - בגלל שאמרו שניתן לצאת מנק הנחה שאין filename גדול מ10 וכל char הוא 1 byte אז הfilename הוא 10 bytes, הsize file וה type file - 4 bytes (כל אחד מהם int) והptrArray בגודל 6 והוא של intים לכן גודלו 24=4*6 bytes.

החישוב של זה הכרחי ויעזור לי בהרבה חישובים בפונקציות

איך כתבתי אותו בזיכרון?

```
// writing empty inode to the memory
void* ptr = myInode;
memset(ptr, '0', STRUCT_SIZE*INT_BLOCK_SIZE);
blkdevsim->write(INT_BLOCK_SIZE*5, STRUCT_SIZE*INT_BLOCK_SIZE, (char*)ptr); // the inode in the second block
```

פונקציית formatה

בתהליך הפורמט מתבצעים שלושה דברים:

א. נכתוב ל-superblock את המידע הנדרש כדי שמערכת ההפעלה תוכל לזהות את המערכת קבצים ולעבוד מולה.

```
blkdevsim->write(0, sizeof(header), (const char*)&header);
// TODO: put your format code here
/*Block size: the operating system needs to know the size of each block that the file system uses
In it, because this number is necessary for the calculations of the various locations
Number of blocks: The os must know how many blocks are in the file system
To be able to save files correctly. If the os is not aware that they exist,
For example, only 254 blocks, it may ask the physical device to store information in the 254th block*
blkdevsim->write(sizeof(header), sizeof(BLOCK_SIZE), BLOCK_SIZE);  // writing the size of each block
blkdevsim->write((sizeof(header) + sizeof(BLOCK_SIZE)), sizeof(AMOUNT_OF_BLOCKS), AMOUNT_OF_BLOCKS);
```

ב. רשימת blocktable ריק (בעל ערכי "0") במקום הנכון בזיכרון.

```
// the block table
void* ptr2 = blockTable;
memset(ptr2, '0', INT_BLOCK_SIZE*4);
blkdevsim->write(INT_BLOCK_SIZE, INT_BLOCK_SIZE*4, (char*)ptr2);
```

ג. במקרה של מימוש תיקיות – אתחול של תיקיית הroot. לכל תיקיה מוגדר מצביע לתיקיה ומצביע לתיקיית האם (המצביע של תיקיית האם זה תיקיית האם). בתהליך האתחול המערכת קבצים צריכה ליצור את המצביעים המתאימים.

פונקציית הcreate_file

```
void create_file(std::string path_str, bool directory)
```

תמיכה בtouch, בפונקציה זו עברתי על הblocktable וחיפשתי בלוק ריק (0) כשמצאתי עידכנתי בblocktable שהבלוק תפוס עכשיו והוספתי את הבלוק לinode בptrArray ואת השם בfilename ואת סוג הקובץ. (ברגע היצירה הקובץ הוא 0 ביטים ולכן לא כתבתי את זה)

<u>פונקציות עזר</u>

```
std::array<int, 6> MyFs::extractBlocksOfFile(std::string filename)
```

פונקציה זו מחזירה מערך בגודל 6 (המקסימום בלוקים שאפשר לחלק בפרגמנטציה) של איך חילקתי את הקובץ בזיכרון (הptrArray) והיא עוזרת לי ממש בפונקציה של לערוך את הקובץ ובפונקציה של להציג את התוכן של הקובץ

```
bool MyFs::isFilenameAlreadyExist(std::string filename)
```

פונקציה זו בודקת האם כבר קיים קובץ בשם שביקש המשתמש – אם כן מודיעים למשתמש שאי אפשר ליצור קובץ בשם שהוא בחר ומבקשים ממנו לנסות שם אחר.

```
int extractSizeFileFromName(std::string filename);
```

מחלצת את הגודל של הקובץ באמצעות שם הקובץ
היא רצה על הinode table ומשווה כל הזמן את הfilename ואם היא מוצאת שם זהה אז היא קוראת את גודל הקובץ (נמצא 14 ביטים לאחר תחילת הnode)

```
std::array<std::string, MAX_BLOCKS_TO_FILE> splitString(std::string str, int limit, int chunk_size);
```

פונקציה שמפצלת את התוכן שמשתמש רוצה להכניס לקובץ למערך שכל תא בו הוא בגודל 256 (בעצם זה חלק מהותי בתהליך הפרגמנטציה)

<u>פונקציית set_content</u>

```
void MyFs::set_content(std::string path_str, std::string content)
```

הפונקציה מוסיפה תוכן חדש (אם הקובץ ריק אז שמה) לתוך קובץ קיים. היא עושה את כל תהליך הפרגמנטציה ואחראית לזה.

<u>הסבר על המימוש שלה:</u>

בהתחלה חישבתי גם בלוקים מלאים צריך בשביל התוכן ומה הגודל של הבלוק האחרון (יכול להיות מלא\חלקי), יצרתי מערך של הבלוקים שיש כבר לקובץ (במידה ויש כבר תוכן לקובץ והשתמשתי בפונקציית עזר שציינתי למעלה) כאשר הסוף שלו מציין תא במערך שיש בו את הספרה 999, לאחר מכן עידכנתי את הטבלת בלוקים (הוספתי את הכמות בלוקים שצריך בשביל התוכן שהמשתמש מבקש) ואז עידכנתי את הptrArray שנמצא בתוך הinode ואז עשיתי את הפרגמנטציה.

יצרתי פונקציית עזר splitContent שמחזירה לי מערך של סטרינגים שכל סטרינג בגודל 256 (הגודל של כל בלוק אצלי) מלבד הסטרינג האחרון שהוא בגודל של הבלוק האחרון שלי (תלוי מה גודל התוכן שהמשתמש מבקש) אחכ עשיתי לולאה שרצה ככמות הבלוקים המלאים שאני צריכה לכתוב אליהם ופשוט כתבתי לפי הבלוקים שכתבתי קודם לכו ומחוץ ללולאה הוספתי את הבלוק האחרון

<u>פונקציית get_content()</u>

```
std::string MyFs::get_content(std::string path_str)
```

הפונקציה בעצם מדפיסה תוכן של הקובץ לפי שם הקובץ.

<u>הסבר על המימוש:</u>

גם פה עשיתי מערך של הבלוקים באמצעות הפונקציית עזר ומשתנים שמייצגים לי כמה בלוקים מלאים יש לי ואת הגודל של הבלוק האחרון. עשיתי לולאה כגודל המקסימום בלוקים שיכול להיות לקובץ (6) קראתי כל פעם לפי הבלוקים ושירשרתי לסטרינג ולבסוף החזרתי אותו.